

Blackthorn

Security Review For Morpho

Collaborative Audit Prepared For: **Morpho**

Lead Security Expert(s): **0x73696d616f**

pkqs90

xiaoming90

Date Audited: **September 17 - September 18, 2026**

Introduction

PriceRatifierV1 and RateRatifierV1 are ratifier contracts that let a maker authorize a whole tree of Midnight offers at once. The maker ratifies the tree's Merkle root, either by direct call or by signature, and an offer can then be taken with a proof, optionally restricted to a single allowed taker. RateRatifierV1 differs in that the maker commits to a simple interest rate rather than a fixed price.

Scope

Repository: [morpho-org/midnight](https://github.com/morpho-org/midnight)

Audited Commit: [531ab3b23f95dcc660b8805e46f089248c6b994e](#)

Final Commit: [bc8be43c7fbcd54831701a23270c42e950e96ed8](#)

Files:

- [src/ratifiers/interfaces/IPriceRatifierV1.sol](#)
- [src/ratifiers/interfaces/IRateRatifierV1.sol](#)
- [src/ratifiers/interfaces/IRatifiersV1Common.sol](#)
- [src/ratifiers/libraries/HashLib.sol](#)
- [src/ratifiers/PriceRatifierV1.sol](#)
- [src/ratifiers/RateRatifierV1.sol](#)

Final Commit Hash

[bc8be43c7fbcd54831701a23270c42e950e96ed8](#)

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.

- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	0	2

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Security Experts Dedicated to This Review

@0x73696d616f

A handwritten signature in white ink on a black background. The signature is stylized, starting with a large 'S' and ending with the name 'Simao' in a cursive script.

@pkqs90

A handwritten signature in white ink on a black background. The signature is highly stylized and cursive, with the letters 'pkqs90' integrated into the flow of the script.

@xiaoming90

A handwritten signature in white ink on a black background. The signature is written in a cursive style, with the name 'Xiaoming90' clearly visible.

Issue L-1: A revoked offer root can be reactivated [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit2/2026-09-morpho-rate-ratifiers-sept-17th-2026/issues/7>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`setIsRootRatified()` leaves `rootNonce` unchanged in `PriceRatifierV1` and `RateRatifierV1`, so an unused approval signature can reactivate a root after its maker revokes it.

Root cause

Both ratifiers allow direct and signed root updates. The direct setter changes the status without advancing the nonce. The signed setter still accepts an earlier signature using that nonce.

Source: [PriceRatifierV1.setIsRootRatified\(\)](#)

```
function setIsRootRatified(address maker, bytes32 root, bool newIsRootRatified)
↳ external returns (bytes32) {
    require(maker == msg.sender || IMidnight(MIDNIGHT).isAuthorized(maker,
↳ msg.sender), Unauthorized());
    ratification[maker][root].isRootRatified = newIsRootRatified;
    emit SetIsRootRatified(msg.sender, maker, root, newIsRootRatified);
    return SET_IS_ROOT_RATIFIED_SUCCESS;
}
```

Attack Path

1. The maker enables root `R` directly and also signs an unused approval of `R` at nonce `0` with a future deadline. In `Midnight`, the maker has authorized the ratifier and a relay contract that forwards signed requests but does not expose arbitrary root updates.

2. The maker calls `setIsRootRatified()` with `false` to revoke `R`. The root's `rootNonce` remains 0.
3. A holder submits the unused signature through the relay. Its call to `setIsRootRatifiedWithSig()` restores `R` because nonce 0 is still current.

Impact

Offers under a revoked root may become executable again if an authorized relay accepts an unexpired approval signature.

Discussion

albert-llimos

Known and intended, raised and closed during internal review. No change planned.

Makers are expected to use either `setIsRootRatified` or `setIsRootRatifiedWithSig` for ratifying a root. A maker who does use both and holds an outstanding signature should revoke the root via `setIsRootRatifiedWithSig`, which advances the nonce and invalidates the signature. That is the signature-cancellation mechanism.

Issue L-2: Rate in the rate ratifier doesn't take the continuous fee into account [RESOLVED]

Source: <https://github.com/sherlock-audit2/2026-09-morpho-rate-ratifiers-sept-17th-2026/issues/8>

Summary

The `RateRatifierV1::isRatified()` function explicitly checks the interest rate, but does not include the continuous fee, which is charged whenever the maker ends up having positive credit, as per `Midnight::take()`.

Vulnerability Detail

In `Midnight::take()` a continuous fee is added on lenders:

```
uint256 buyerCreditIncrease = UtilsLib.zeroFloorSub(units, buyerPos.debt);
uint256 sellerCreditDecrease = UtilsLib.min(units, sellerPos.credit);
uint256 sellerDebtIncrease = units - sellerCreditDecrease;
uint128 buyerPendingFeeIncrease =
    UtilsLib.toUint128(buyerCreditIncrease.mulDivDown(_marketState.continuousFee *
    ↳ timeToMaturity, WAD));
```

But the rate that is checked in the rate ratifier doesn't include it:

```
function isRatified(Offer memory offer, bytes memory ratifierData, address taker)
    ↳ external view returns (bytes32) {
    ...
    uint256 timeToMaturity = UtilsLib.zeroFloorSub(offer.market.maturity,
    ↳ block.timestamp);
    uint256 offerPrice = TickLib.tickToPrice(offer.tick);
    if (offer.buy) require(offerPrice <= WAD.mulDivDown(WAD, WAD + rate *
    ↳ timeToMaturity), WorsePrice());
```

```
    else require(offerPrice >= WAD.mulDivUp(WAD, WAD + rate * timeToMaturity),  
        ↪ WorsePrice());  
    ...  
}
```

Impact

Users of the rate ratifier can be misled since their real interest rate will be lower than the one they sign if they have credit.

Code Snippet

https://github.com/sherlock-audit2/2026-09-morpho-rate-ratifiers-sept-17th-2026/blob/main/morpho-org__midnight/src/Midnight.sol#L413-L417

Tool Used

Manual Review

Recommendation

Add a comment in the natpsec of the rate ratifier contract to confirm the rate doesn't include the continuous fee.

Discussion

albert-llimos

Acknowledged and addressed in [PR#1205](#).

Disclaimers

Blackthorn does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.